

Olist E-Commerce Dataset – Database Setup and Verification Report

Samar Abo Alouf

1. Project Objective

The main objective of this task was to understand the Olist e-commerce dataset, load the dataset into a relational database, and verify that the database works correctly by running SQL queries and joining related tables.

The main problem that will be studied in later stages of the project is:

Can we determine whether an order will be delivered on time or late?

2. Dataset Understanding

The first step was to review the provided Olist Dataset Introduction PDF and then access the Olist dataset page on Kaggle.

The dataset structure and the purpose of the main tables were reviewed. Special attention was given to the main identifiers used to connect the tables:

- order_id
- customer_id
- product_id
- seller_id

The relationships between the tables were also reviewed to understand how customer, order, product, seller, payment, and delivery information are connected.

Public Kaggle notebooks related to the Olist dataset were also reviewed to understand common approaches and possible ideas for working with the dataset, without copying existing work.

3. Downloading the Dataset

The Olist dataset was downloaded from Kaggle as CSV files.

The following files were used:

- olist_customers_dataset.csv
- olist_orders_dataset.csv
- olist_order_items_dataset.csv
- olist_order_payments_dataset.csv
- olist_order_reviews_dataset.csv
- olist_products_dataset.csv
- olist_sellers_dataset.csv

- olist_geolocation_dataset.csv
- product_category_name_translation.csv

4. PostgreSQL and Docker Setup

Docker Desktop was used to run PostgreSQL locally.

The PostgreSQL Docker image was downloaded, and a PostgreSQL container was created and started.

A PostgreSQL server was then configured with:

- PostgreSQL running inside Docker
- Port 5432
- PostgreSQL user: postgres
- A local PostgreSQL database named olist

pgAdmin 4 was also installed and run using Docker to provide a graphical interface for managing and viewing the PostgreSQL database.

The pgAdmin server was successfully connected to the PostgreSQL container.

```
PS C:\Users\rosto> docker start pg-tutorial
pg-tutorial
PS C:\Users\rosto> docker start pgadmin-tutorial
pgadmin-tutorial
PS C:\Users\rosto> docker ps
CONTAINER ID   IMAGE          COMMAND                  CREATED        STATUS        PORTS
e3c6b988b6cc   dpape/pgadmin4 "/entrypoint.sh"        2 days ago    Up 6 seconds  0.0.0.0:5050->80/tcp, [::]:5050->80/tcp
5b37aa2ec306   pgadmin-tutorial "docker-entrypoint.s..." 2 days ago    Up 16 seconds  0.0.0.0:5432->5432/tcp, [::]:5432->5432/tcp
PS C:\Users\rosto>
```

Figure 1: PostgreSQL and pgAdmin containers running successfully in Docker.

5. Loading the CSV Files into PostgreSQL

The Olist CSV files were copied from the local computer into the PostgreSQL Docker container.

A folder named:

```
| /olist_data
```

was used inside the container to store the CSV files.

The database tables were then created inside the olist database.

The following tables were created:

- customers
- orders
- order_items
- order_payments
- order_reviews
- products
- sellers

- geolocation
- product_category_translation

The PostgreSQL \copy command was used to import the CSV data into the corresponding tables.

The import was completed successfully. For example:

- customers: 99,441 rows
- orders: 99,441 rows
- order_items: 112,650 rows
- order_payments: 103,886 rows
- order_reviews: 99,224 rows
- products: 32,951 rows
- sellers: 3,095 rows
- geolocation: 1,000,163 rows
- product_category_translation: 71 rows

This confirmed that the Olist dataset was successfully stored in PostgreSQL as relational tables.

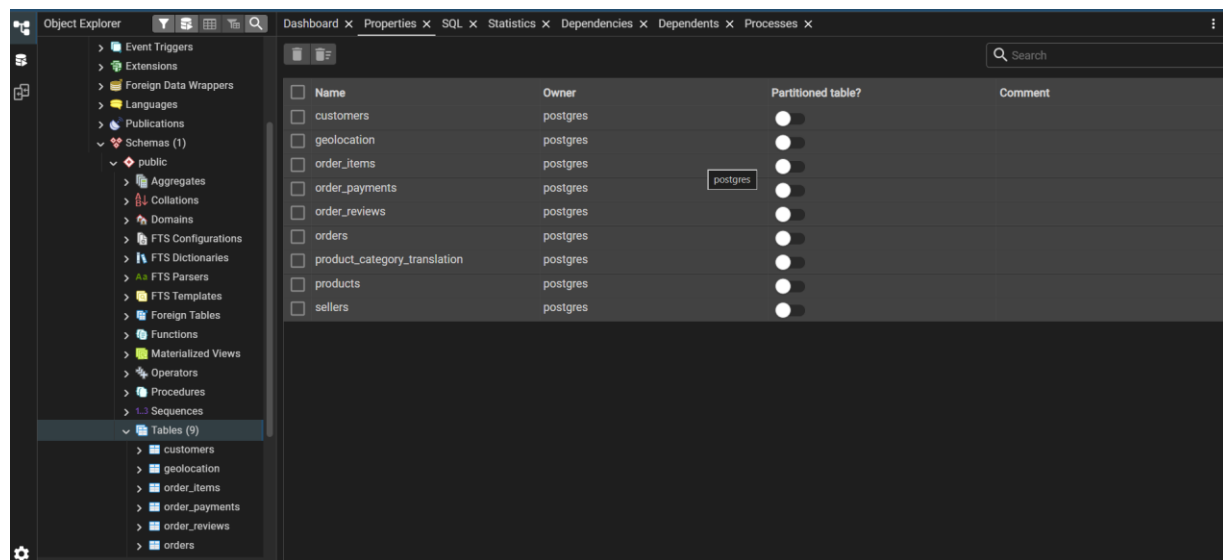


Figure 2: Olist relational tables successfully created and displayed in pgAdmin.

6. Database Testing and Verification

After loading the data, the PostgreSQL database was accessed through the terminal using psql.

Several SQL queries were executed to confirm that the data could be read correctly.

For example, customer records were checked using:

```
SELECT * FROM customers LIMIT 5;
```

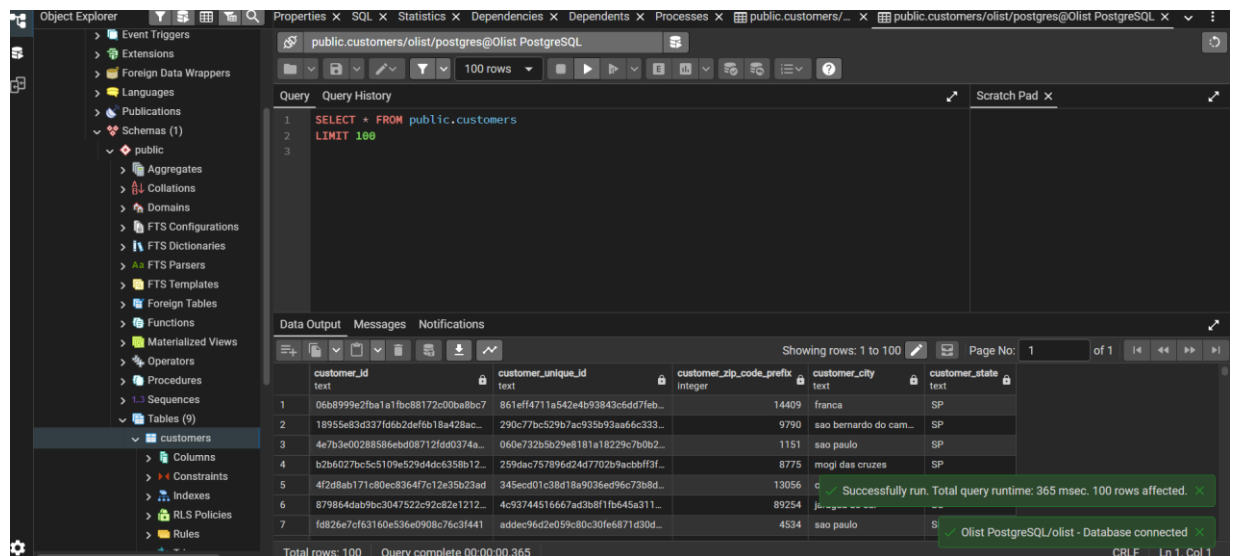


Figure 3: Sample records from the customers table displayed successfully in pgAdmin.

The number and distribution of order statuses were also checked using:

```
SELECT order_status, COUNT(*)
FROM orders
GROUP BY order_status
ORDER BY COUNT(*) DESC;
```

The query successfully returned the different order statuses and their counts.

```
olist=# SELECT order_status, COUNT(*)
FROM orders
GROUP BY order_status
ORDER BY COUNT(*) DESC;
 order_status | count
-----+-----
 delivered    | 96478
 shipped       | 1107
 canceled     | 625
 unavailable   | 609
 invoiced     | 314
 processing    | 301
 created       | 5
 approved     | 2
(8 rows)

olist=#
```

Figure 4: SQL query showing the distribution of order statuses in the Olist database.

7. Testing a JOIN Between Tables

A JOIN was performed between the orders and customers tables using customer_id.

```
SELECT
o.order_id,
o.order_status,
c.customer_city,
c.customer_state
```

```
FROM orders o
JOIN customers c
ON o.customer_id = c.customer_id
LIMIT 10;
```

The query successfully combined information from both tables and returned:

- Order ID
- Order status
- Customer city
- Customer state

This confirmed that the tables could be connected and queried together successfully.

```
olist=# SELECT
  o.order_id,
  o.order_status,
  c.customer_city,
  c.customer_state
FROM orders o
JOIN customers c
ON o.customer_id = c.customer_id
LIMIT 10;
```

order_id	order_status	customer_city	customer_state
e481f51cbdc54678b7cc49136f2d6af7	delivered	sao paulo	SP
53cdb2fc8bc7dce0b6741e2150273451	delivered	barreiras	BA
47770eb9100c2d0c44946d9cf07ec65d	delivered	vianopolis	GO
949d5b44dbf5de918fe9c16f97b45f8a	delivered	sao goncalo do amarante	RN
ad21c59c0840e6cb83a9ceb5573f8159	delivered	santo andre	SP
a4591c265e18cb1dcee52889e2d8acc3	delivered	congonhas	PR
9d531c565e28c3e0d756192f84d8731f	delivered	marica	RJ
4da3bd761ca73d692ae0f5c252495546	delivered	rio de janeiro	RJ
80d817467dc3d0a4bb5ab8986c962b1f	delivered	brasilia	DF
c75acba15d2db29345327974641467d	delivered	rio de janeiro	RJ

```
(10 rows)
olist=#
```

Figure 5: JOIN between the orders and customers tables executed successfully in PostgreSQL.

8. Understanding the Target Problem

The future analysis will focus on determining whether an order was delivered late or on time.

The main fields related to this problem are:

- order_delivered_customer_date
- order_estimated_delivery_date

By comparing the actual delivery date with the estimated delivery date, each delivered order can later be classified as:

- On Time
- Late

No full Exploratory Data Analysis (EDA) was performed at this stage because it is not required for the current task.

9. Final Result

The task was completed successfully.

At the end of the task:

- The Olist dataset was understood and reviewed.
- The CSV files were downloaded from Kaggle.
- PostgreSQL was successfully configured using Docker.
- pgAdmin was connected to PostgreSQL.
- The Olist data was loaded into relational database tables.
- The database was accessed through the terminal.
- SQL queries were executed successfully.
- A JOIN between two tables was tested successfully.
- The main relationships between the tables were understood.
- The late/on-time delivery problem was clearly identified.

The PostgreSQL database is now running locally and is ready for the next stages of the project.

1. هدف المشروع

كان الهدف الأساسي من هذه المهمة هو فهم بيانات Olist E-Commerce Dataset، ثم تحميلها إلى قاعدة بيانات علائقية، وبعد ذلك التأكد من أن قاعدة البيانات تعمل بشكل صحيح من خلال تشغيل استعلامات SQL وتجربة الربط بين الجداول. المشكلة الرئيسية التي سيتم العمل عليها في المراحل القادمة هي: هل يمكن تحديد ما إذا كان الطلب سيصل في الوقت المحدد أم سيتأخر؟

2. فهم البيانات

في البداية تمت مراجعة ملف Olist Dataset Introduction PDF المرفق، وبعد ذلك تمت زيارة صفحة الداتا سيت على Kaggle. تمت مراجعة الجداول وفهم وظيفة كل جدول، مع التركيز بشكل خاص على أهم المعرفات المستخدمة لربط الجداول ببعضها:

- order_id
- customer_id
- product_id
- seller_id

كما تمت مراجعة العلاقات بين الجداول لفهم كيفية ارتباط بيانات العملاء والطلبات والمنتجات والبائعين وعمليات الدفع والتوصيل ببعضها. تم أيضًا الاطلاع على بعض الـ Public Notebooks الخاصة بنفس الداتا سيت على Kaggle للاستفادة من الأفكار والأساليب المستخدمة، بدون نسخ العمل الموجود فيها.

3. تحميل الداتا سيت

تم تحميل ملفات Olist من Kaggle بصيغة CSV. الملفات التي تم استخدامها هي:

- olist_customers_dataset.csv
- olist_orders_dataset.csv
- olist_order_items_dataset.csv
- olist_order_payments_dataset.csv
- olist_order_reviews_dataset.csv
- olist_products_dataset.csv
- olist_sellers_dataset.csv
- olist_geolocation_dataset.csv
- product_category_name_translation.csv

4. إعداد PostgreSQL باستخدام Docker

تم استخدام Docker Desktop لتشغيل PostgreSQL محليًا على الجهاز.

تم تنزيل PostgreSQL Image من Docker، وبعد ذلك تم إنشاء وتشغيل PostgreSQL Container.

تم إعداد PostgreSQL بحيث يعمل داخل Docker باستخدام:

- PostgreSQL داخل Docker

- Port رقم 5432

- اسم المستخدم postgres

- قاعدة بيانات محلية باسم olist

كما تم تشغيل pgAdmin 4 باستخدام Docker للحصول على واجهة رسومية تساعد في إدارة وعرض قاعدة البيانات.

تم ربط pgAdmin مع PostgreSQL بنجاح.

5. إدخال ملفات CSV إلى PostgreSQL

تم نقل ملفات CSV من الجهاز إلى PostgreSQL Container الموجود داخل Docker.

تم استخدام مجلد داخل ال-Container باسم:

```
| /olist_data
```

لتخزين ملفات الداتا.

بعد ذلك تم إنشاء الجداول داخل قاعدة البيانات olist.

الجدول التي تم إنشاؤها هي:

- customers

- orders

- order_items

- order_payments

- order_reviews

- products

- sellers

- geolocation

- product_category_translation

بعد إنشاء الجداول، تم استخدام أمر PostgreSQL:

```
| \copy
```

لتحميل بيانات كل ملف CSV إلى الجدول المناسب.

تمت عملية الإدخال بنجاح، ومن أمثلة أعداد الصفوف التي تم تحميلها:

- customers: عدد 99,441 صف

- orders: عدد 99,441 صف

- order_items: عدد 112,650 صف

- order_payments: عدد 103,886 صف

- order_reviews: عدد 99,224 صف
- products: عدد 32,951 صف
- sellers: عدد 3,095 صف
- geolocation: عدد 1,000,163 صف
- product_category_translation: عدد 71 صف

وهذا أكد أن بيانات Olist تم إدخالها بنجاح إلى PostgreSQL على شكل جداول علائقية.

6. تجربة قاعدة البيانات والتأكد من عملها

بعد رفع البيانات، تم الاتصال بقاعدة بيانات PostgreSQL من خلال الـ Terminal باستخدام psql.

تم تشغيل عدة استعلامات للتأكد من أن البيانات موجودة ويمكن قراءتها بشكل صحيح.

على سبيل المثال، تم عرض أول خمس سجلات من جدول العملاء باستخدام:

```
SELECT * FROM customers LIMIT 5;
```

كما تم تشغيل Query لمعرفة حالات الطلبات وعدد كل حالة:

```
SELECT order_status, COUNT(*)
FROM orders
GROUP BY order_status
ORDER BY COUNT(*) DESC;
```

وتم الحصول على النتائج بنجاح.

7. تجربة JOIN بين جداولين

تم تنفيذ JOIN بين جدول orders وجدول customers باستخدام العمود المشترك customer_id.

```
SELECT
o.order_id,
o.order_status,
c.customer_city,
c.customer_state
FROM orders o
JOIN customers c
ON o.customer_id = c.customer_id
LIMIT 10;
```

أظهر الاستعلام معلومات من الجدولين في نفس النتيجة، ومنها:

- رقم الطلب
- حالة الطلب
- مدينة العميل
- ولاية العميل

وهذا أكد أن الربط بين الجداول يعمل بشكل صحيح وأن البيانات يمكن التعامل معها كقاعدة بيانات علائقية.

8. فهم المشكلة التي سيتم العمل عليها

المشكلة التي سيتم تحليلها في المراحل القادمة هي تحديد ما إذا كان الطلب تم توصيله متأخرًا أو في الوقت المحدد.

أهم الأعمدة المرتبطة بهذه المشكلة هي:

- order_delivered_customer_date

- order_estimated_delivery_date

من خلال مقارنة تاريخ التوصيل الفعلي بتاريخ التوصيل المتوقع يمكن لاحقًا تصنيف كل طلب إلى:

- On Time – في الوقت المحدد

- Late – متأخر

لم يتم إجراء EDA كامل في هذه المرحلة لأنه غير مطلوب في هذا التأسك وسيتم العمل عليه في مرحلة لاحقة.

9. النتيجة النهائية

تم إنجاز المهمة بنجاح.

في نهاية العمل تم:

- فهم ومراجعة Olist Dataset.

- تحميل ملفات CSV من Kaggle.

- إعداد PostgreSQL باستخدام Docker.

- ربط PostgreSQL مع pgAdmin.

- إنشاء قاعدة بيانات olist.

- إدخال ملفات CSV إلى جداول علانقية داخل PostgreSQL.

- الاتصال بقاعدة البيانات من خلال الـ Terminal.

- تشغيل عدة SQL Queries بنجاح.

- تنفيذ JOIN بين جدولين بنجاح.

- فهم العلاقات الأساسية بين الجداول.

- فهم مشكلة التنبؤ بوصول الطلب متأخرًا أو في الوقت المحدد.

قاعدة البيانات الآن تعمل محليًا وأصبحت جاهزة للمراحل القادمة من المشروع.